



Intelligenza Artificiale: Metodi e Applicazioni

Progettazione, sperimentazione e confronto di algoritmi di machine learning per la guida autonoma in TORCS

Gruppo 25

Vittorio Postiglione - Matricola: 0612707620

Mattia Sanzari - Matricola: 0612707696

Mario Zito - Matricola: 0612708073

Docenti:

Prof. Mario Vento

Prof.ssa Alessia Saggese

Anno Accademico 2024-2025

Indice

1	Introduzione	4
1.1	Scelte Progettuali	4
1.2	Obiettivi della Sperimentazione	4
2	Analisi del Problema	5
2.1	Caratteristiche del Dominio TORCS	5
2.2	Sfide Tecniche	5
2.3	Requisiti del Sistema	6
2.4	Approccio Metodologico	6
3	Architettura del Sistema	6
3.1	Progetto Base di Partenza	6
3.2	Architettura del Progetto Attuale	7
3.2.1	Moduli Aggiunti	7
3.2.2	Architettura KNN Regressivo	7
3.2.3	Architettura KNN Classificatorio	8
3.2.4	Confronto tra Approcci KNN	9
3.2.5	Componenti di Supporto	9
3.3	Pipeline dei Dati	10
3.4	Gestione delle Performance	10
3.5	Sistema di Menu e Automazione	10
3.5.1	Menu Principale (torcs_menu.bat)	11
3.5.2	Script di Automazione	11
3.6	Sistema di Guida Manuale	11
3.6.1	HumanController	11
3.6.2	Gestione Input Avanzata	12
3.6.3	EnhancedDataCollectionManager	12
3.7	Interfacce e Protocolli	12
4	Dataset e Preprocessing	12
4.1	Architettura dei Dataset	12
4.1.1	Enhanced Dataset	13
4.1.2	Human Dataset	14
4.1.3	Considerazioni sui Sensori Utilizzati	14
4.2	Pipeline di Conversione	15
4.3	Preprocessing dei Dati	16
4.4	Sfide del Bilanciamento del Dataset	16
4.4.1	Problematiche Legate ai Circuiti	16
4.4.2	Impatto sui Modelli di Machine Learning	16
4.4.3	Strategie di Bilanciamento Implementate	17
4.5	Sistema di Analisi Python	17
4.5.1	Sistema di Fine Tuning	18

4.5.2	Librerie Utilizzate	18
5	Metodologia di Sperimentazione	18
5.1	Approccio Sperimentale	18
5.2	Analisi delle Feature	18
5.3	Problematiche Iniziali del KNN	19
6	Risultati	20
6.1	Analisi della Distribuzione Iniziale	20
6.2	Influenza delle Caratteristiche del Tracciato	20
6.3	Strategia di Bilanciamento Adottata	21
6.4	Ottimizzazione Iterativa del KNN	21
6.4.1	Confronto tra modelli KNN	21
6.5	Limitazioni Hardware	23
6.6	Sviluppo del Modello MLP	23
6.7	Bilanciamento Dataset per MLP	23
6.8	Addestramento MLP	24
7	Analisi delle Performance	26
7.1	Performance Computazionali del KNN	26
7.2	Ottimizzazioni Implementate	27
7.3	Confronto KNN vs MLP	27
8	Discussione	27
8.1	Impatto del Bilanciamento del Dataset	27
8.2	Limitazioni e Trade-off	28
8.3	Approcci Metodologici	28
9	Conclusioni	28
9.1	Valutazione Comparativa dei Modelli	28
9.2	Considerazioni Finali	29

1 Introduzione

La presente relazione documenta lo sviluppo e il confronto di diversi algoritmi di machine learning per la realizzazione di un sistema di guida autonoma nel simulatore TORCS (The Open Racing Car Simulator).

L'obiettivo principale di questa relazione è presentare l'implementazione, la valutazione e il confronto di tre approcci distinti di classificazione e regressione per il controllo automatico di un veicolo da corsa.

Questa relazione nasce dalla necessità di documentare l'esplorazione delle potenzialità di diversi paradigmi di apprendimento automatico applicati al controllo di sistemi dinamici complessi. In particolare, il dominio della guida autonoma presenta sfide significative che richiedono decisioni in tempo reale basate su informazioni sensoriali multiple e variabili.

1.1 Scelte Progettuali

Le scelte progettuali effettuate per questo progetto sono state guidate dalla volontà di confrontare approcci fondamentalmente diversi nel panorama del machine learning:

- **K-Nearest Neighbors (KNN) Classificatore:** Implementazione di un approccio basato su classificazione discreta delle azioni di guida, dove il comportamento del veicolo è categorizzato in un insieme finito di azioni predefinite.
- **K-Nearest Neighbors (KNN) Regressivo:** Utilizzo dello stesso paradigma di apprendimento lazy, ma applicato alla regressione per ottenere valori continui di controllo (sterzo, accelerazione, frenata).
- **Multi-Layer Perceptron (MLP) Regressivo:** Implementazione di una rete neurale feed-forward per l'apprendimento di pattern complessi e non lineari nei dati di guida.

Queste scelte permettono di confrontare:

- **Classificazione vs regressione:** Azioni discrete vs controlli continui
- **Semplicità vs complessità:** Algoritmi interpretabili vs modelli più sofisticati

1.2 Obiettivi della Sperimentazione

La sperimentazione condotta mira a:

1. **Valutare l'efficacia** di ciascun approccio nel contesto specifico della guida autonoma in TORCS
2. **Identificare i punti di forza e le limitazioni** di ogni algoritmo implementato
3. **Determinare la configurazione ottimale** dei parametri per ciascun modello
4. **Fornire una base empirica** per la scelta del modello definitivo

Il processo di sperimentazione include analisi quantitative delle performance, valutazione qualitativa del comportamento di guida, e considerazioni pratiche relative all'implementazione e al deployment dei diversi approcci.

La metodologia adottata prevede un confronto sistematico basato su metriche oggettive di performance e valutazione dell'efficienza computazionale, al fine di identificare la soluzione più adatta alle specifiche esigenze del progetto.

2 Analisi del Problema

La guida autonoma in ambiente simulato presenta una serie di sfide tecniche e metodologiche che richiedono un'analisi approfondita per la progettazione di una soluzione efficace.

2.1 Caratteristiche del Dominio TORCS

TORCS (The Open Racing Car Simulator) rappresenta un ambiente di simulazione complesso caratterizzato da:

- **Vincoli temporali:** Necessità di decisioni in tempo reale con latenze minime
- **Spazio degli stati continuo:** Infinite possibili combinazioni di posizione, velocità, orientamento e condizioni ambientali

2.2 Sfide Tecniche

L'implementazione di un sistema di guida autonoma in questo contesto presenta diverse criticità:

1. **Rappresentazione dello stato:** Definizione di un insieme appropriato di features che catturi le informazioni rilevanti dall'ambiente sensoriale
2. **Spazio delle azioni:** Scelta tra rappresentazione discreta (classificazione) o continua (regressione) delle azioni di controllo
3. **Dispositivi di input per la raccolta dati:** Gestione delle differenze tra dispositivi di controllo discreti (tastiera: valori binari premuto/non premuto) e continui (gamepad: valori analogici intermedi), che influenzano significativamente la natura e la qualità dei dati di addestramento
4. **Generalizzazione:** Capacità del modello di adattarsi a situazioni non viste durante l'addestramento
5. **Efficienza computazionale:** Bilanciamento tra accuratezza del modello e velocità di inferenza
6. **Robustezza:** Gestione di situazioni anomale e comportamenti di recovery

2.3 Requisiti del Sistema

Basandosi sull'analisi del dominio, il sistema deve soddisfare i seguenti requisiti:

- **Performance di guida:** Capacità di completare giri di pista in modo efficiente e sicuro
- **Reattività:** Tempo di risposta compatibile con i vincoli real-time del simulatore
- **Stabilità:** Comportamento consistente e prevedibile in diverse condizioni
- **Adattabilità:** Capacità di gestire variazioni nelle condizioni di pista
- **Interpretabilità:** Possibilità di analizzare e comprendere le decisioni del sistema

2.4 Approccio Metodologico

Per affrontare queste sfide, la strategia adottata prevede:

- **Confronto multi-paradigma:** Valutazione di approcci diversi (classificazione vs regressione)
- **Analisi empirica:** Sperimentazione sistematica con dataset di guida umana
- **Ottimizzazione parametrica:** Tuning dei parametri per massimizzare le performance
- **Valutazione multidimensionale:** Considerazione di metriche diverse (accuratezza, velocità, stabilità)

3 Architettura del Sistema

Il progetto attuale è stato sviluppato partendo dal framework base fornito dall'università al quale sono state apportate significative modifiche ed estensioni per implementare i diversi algoritmi di machine learning.

3.1 Progetto Base di Partenza

Il framework originale fornito dall'università presenta una struttura modulare composta da:

- **Controller:** Classe astratta che definisce l'interfaccia base per tutti i driver
- **SimpleDriver:** Implementazione di riferimento con logica di guida basata su regole euristiche
- **Action:** Struttura dati per rappresentare i comandi di controllo del veicolo
- **SensorModel:** Interfaccia per l'accesso ai dati sensoriali del simulatore
- **Client:** Gestione della comunicazione con il server TORCS

Il `SimpleDriver` originale implementa una strategia di guida deterministica basata su:

- Calcolo dell'angolo di sterzata basato sulla posizione rispetto al centro pista
- Gestione della velocità in funzione della geometria della pista
- Sistema di cambio marcia automatico basato sui giri del motore
- Logica di recupero per situazioni di blocco del veicolo
- Filtro ABS per ottimizzare la frenata

3.2 Architettura del Progetto Attuale

Partendo da questa base, il progetto è stato esteso con l'implementazione di tre approcci di machine learning distinti, mantenendo la compatibilità con l'architettura originale:

3.2.1 Moduli Aggiunti

1. **KNNDriver**: Implementazione dell'algoritmo K-Nearest Neighbors per la regressione
 - Utilizza un KD-tree per ricerca efficiente dei vicini
 - Supporta normalizzazione dei dati e configurazione parametri
 - Predice valori continui per sterzo, accelerazione e frenata
2. **KNNClassifierDriver**: Implementazione KNN per la classificazione discreta
 - Classifica le azioni in categorie predefinite (DrivingAction)
 - Utilizza ActionDiscretizer per convertire azioni continue in discrete
 - Implementa gestione robusta dei casi edge e situazioni fuori pista
3. **MLPDriver**: Interfaccia per rete neurale Multi-Layer Perceptron
 - Comunica con un modello Python tramite socket UDP
 - Mantiene le funzionalità di base (cambio, ABS, frizione) del SimpleDriver
 - Delega le decisioni di sterzo, accelerazione e frenata alla rete neurale

3.2.2 Architettura KNN Regressivo

Il **KNNDriver** implementa un approccio regressivo per la predizione di valori continui di controllo:

Caratteristiche Tecniche

- **Struttura Dati**: Utilizza **KDTree** con nodi **DataPoint** contenenti 14 features normalizzate
- **Algoritmo di Ricerca**: Ricerca efficiente dei K vicini con complessità $O(\log n)$
- **Predizione**: Media pesata basata sulla distanza euclidea inversa per sterzo, accelerazione e freno

- **Gestione Pesì:** Formula $w_i = \frac{1}{d_i + \epsilon}$ dove $\epsilon = 10^{-10}$ evita divisioni per zero
- **Features Input:** 10 sensori di pista selezionati + velocità + angolo + posizione + distanza dalla partenza
- **Output:** Valori continui nell'intervallo $[-1,1]$ per sterzo, $[0,1]$ per accelerazione/freno

Algoritmo di Predizione

L'approccio regressivo utilizza una media pesata per calcolare i valori di controllo:

- Media pesata: $\hat{y} = \frac{\sum_{i=1}^k w_i \cdot y_i}{\sum_{i=1}^k w_i}$ dove $w_i = \frac{1}{d_i + \epsilon}$
- Predizione separata per sterzo, accelerazione e freno
- Gestione automatica marce tramite logica RPM (3000-7500 range)
- Interpolazione che produce azioni intermedie per situazioni non viste

3.2.3 Architettura KNN Classificatorio

Il `KNNClassifierDriver` implementa un approccio classificatorio basato su azioni discrete predefinite:

Caratteristiche Tecniche

- **Struttura Dati:** Utilizza `KDTreeClassifier` con nodi `DataPointClassifier` e classi `DrivingAction`
- **Spazio delle Azioni:** 15 azioni discrete predefinite
- **Categorie Azioni:** Dritte (3), curve sinistra/destra (6), emergenza (3), recupero fuori pista (3)
- **Algoritmo di Voto:** Voto di maggioranza pesato per distanza con accumulo per classe
- **Discretizzazione:** `ActionDiscretizer` converte azioni continue in classi tramite distanza euclidea minima

Algoritmo di Classificazione

L'approccio classificatorio utilizza un sistema di voto pesato:

- Voto di maggioranza pesato: classe = $\arg \max_c \sum_{i \in N_k} w_i \cdot \mathbb{I}(y_i = c)$
- Accumulo voti per classe con peso inversamente proporzionale alla distanza
- Conversione diretta da `DrivingAction` a `Action` tramite valori predefiniti
- Azioni specifiche per recupero fuori pista (`OFFTRACK_LEFT`, `OFFTRACK_RIGHT`, `OFFTRACK_STRAIGHT`)

3.2.4 Confronto tra Approcci KNN

I due modelli KNN presentano caratteristiche distinte che li rendono adatti a diversi scenari operativi:

Differenze Principali

- **Rappresentazione Output:**
 - Regressivo: Valori continui interpolati dai vicini
 - Classificatorio: Selezione di azioni predefinite e validate
- **Gestione Situazioni Critiche:**
 - Regressivo: Gestione implicita tramite dati di recupero inclusi nel dataset di training
 - Classificatorio: Sistema completo con rilevamento fuori pista e strategie di recupero KNN-based
- **Complessità Computazionale:**
 - Regressivo: $K=8$, calcolo media pesata diretta
 - Classificatorio: $K=15$, voto pesato con accumulo per classe
- **Flessibilità:**
 - Regressivo: Adattamento continuo a situazioni intermedie
 - Classificatorio: Comportamenti consistenti tramite azioni predefinite

3.2.5 Componenti di Supporto

Il sistema utilizza diversi componenti condivisi che supportano l'implementazione degli algoritmi di machine learning:

Strutture Dati Condivise

- **DataPoint/DataPointClassifier:** Classi per incapsulare features normalizzate (14 dimensioni) e azioni target, con supporto per calcolo distanza euclidea e thread-safety
- **KDTree/KDTreeClassifier:** Implementazione ottimizzata per ricerca spaziale con complessità $O(\log n)$, utilizzando costruzione ricorsiva e pruning intelligente
- **ActionDiscretizer:** Convertitore per trasformare dataset continui in formato classificatorio tramite mappatura basata su distanza euclidea minima
- **KNNConfig:** Sistema di configurazione centralizzato per parametri K , percorsi dataset e modalità operative

Sistema di Gestione Situazioni Critiche (Solo KNN Classificatorio)

Il sistema di gestione delle situazioni critiche è implementato esclusivamente nel `KNNClassifierDriver`:

- **Rilevamento Fuori Pista:** Monitoraggio automatico tramite sensori di traccia con soglia conservativa (almeno 2 sensori centrali devono rilevare la pista a distanza > 5 metri)
- **Strategia di Recupero:** Utilizza il classificatore KNN per determinare l'azione di recupero appropriata basandosi sui dati di training
- **Evitamento Ostacoli:** Sistema di frenata di emergenza integrato nell'azione di default per distanze frontali < 10 metri

Il `KNNDriver` regressivo gestisce le situazioni critiche in modo implicito attraverso i dati di recupero presenti nel dataset di training, senza un sistema di rilevamento dedicato.

Componenti di Raccolta Dati

- **EnhancedDataCollector:** Sistema avanzato per raccolta dati con supporto gamepad e normalizzazione automatica
- **HumanController:** Driver specializzato per registrazione di sessioni di guida umana

3.3 Pipeline dei Dati

L'architettura implementa una pipeline completa per la gestione dei dati:

1. **Raccolta:** Utilizzo di `HumanController` per registrare sessioni di guida
2. **Preprocessing:** Normalizzazione e discretizzazione dei dati
3. **Training:** Costruzione di strutture dati ottimizzate (KD-tree) o training di reti neurali
4. **Inferenza:** Predizione in tempo reale durante la guida autonoma

3.4 Gestione delle Performance

Per affrontare le sfide computazionali, sono state implementate diverse ottimizzazioni:

- **Precaricamento:** I modelli vengono inizializzati all'avvio per evitare timeout
- **Strutture dati efficienti:** Utilizzo di KD-tree per ricerche $O(\log n)$
- **Normalizzazione:** Preprocessing dei dati per migliorare convergenza e stabilità
- **Configurazione adattiva:** Parametri ottimizzati per diversi scenari di utilizzo

3.5 Sistema di Menu e Automazione

Una delle principali estensioni rispetto al framework originale è l'implementazione di un sistema completo di menu e automazione che semplifica notevolmente l'utilizzo del progetto:

3.5.1 Menu Principale (torcs_menu.bat)

Il framework originale richiedeva l'esecuzione manuale di comandi Java con parametri specifici per ogni operazione. È stato sviluppato un sistema di menu interattivo che organizza tutte le funzionalità in categorie:

- **Raccolta Dati:** Guida manuale con raccolta automatica dei dati
- **Gestione Dataset:** Visualizzazione statistiche, conversione formati, strumenti di analisi
- **Guida Autonoma:** Esecuzione dei diversi algoritmi implementati
- **Documentazione:** Accesso a guide e strumenti di analisi esterni
- **Integrazione TORCS:** Avvio automatico del simulatore

3.5.2 Script di Automazione

Ogni funzionalità è supportata da script dedicati nella cartella `scripts/`:

- `run_manual_driving.bat`: Avvio della modalità di guida manuale
- `run_knn_driving_human.bat`: Esecuzione del KNN regressivo
- `run_knn_classifier.bat`: Esecuzione del KNN classificatore
- `run_mlp_driving_human.bat`: Esecuzione del driver MLP
- `run_simpledriver.bat`: Esecuzione del driver di riferimento

Questi script gestiscono automaticamente:

- Configurazione del classpath Java
- Parametri di connessione al server TORCS
- Gestione degli errori e cleanup dei processi
- Verifica delle dipendenze e dei file necessari

3.6 Sistema di Guida Manuale

Un componente fondamentale aggiunto al progetto è il sistema di guida manuale per la raccolta dei dati di training:

3.6.1 HumanController

Il `HumanController` implementa un'interfaccia completa per la guida manuale che supporta:

- **Input multipli:** Tastiera (WASD, frecce, numpad) e gamepad analogico
- **Controllo real-time:** Gestione degli input senza latenze
- **Assistenze configurabili:** Cambio automatico, ABS, assistenza sterzo
- **Raccolta dati integrata:** Registrazione automatica delle sessioni di guida

3.6.2 Gestione Input Avanzata

Il sistema gestisce le differenze tra dispositivi di input:

- **Tastiera:** Input discreto con smoothing per simulare controlli analogici
- **Gamepad:** Input analogico nativo con zone morte configurabili
- **Sensibilità adattiva:** Parametri ottimizzati per diversi stili di guida

3.6.3 EnhancedDataCollectionManager

La raccolta dati è gestita da un sistema avanzato che:

- Registra sincronizzazione tra stato sensoriale e azioni umane
- Gestisce formati multipli (enhanced e human dataset)
- Implementa controlli di qualità sui dati raccolti
- Fornisce statistiche in tempo reale durante la raccolta

3.7 Interfacce e Protocolli

L'architettura mantiene la compatibilità con il protocollo TORCS originale, estendendolo con:

- **Comunicazione UDP:** Per l'integrazione con componenti Python (MLP)
- **Gestione file CSV:** Per il caricamento e salvataggio dei dataset
- **Logging configurabile:** Per debug e analisi delle performance
- **Integrazione esterna:** Collegamenti a strumenti di analisi (Google Colab)

4 Dataset e Preprocessing

Il progetto utilizza un approccio a doppio dataset per ottimizzare la raccolta e l'utilizzo dei dati di training, implementando una strategia flessibile che separa la raccolta completa dei dati dalla loro elaborazione specifica per i modelli.

4.1 Architettura dei Dataset

Sono stati implementati due formati di dataset distinti, ciascuno con obiettivi specifici:

4.1.1 Enhanced Dataset

L'enhanced_dataset.csv rappresenta il dataset completo che raccoglie tutti i sensori disponibili in TORCS che potrebbero essere potenzialmente utili durante lo sviluppo e la selezione delle feature per i modelli di machine learning.

Sensori memorizzati (totale: 35+ campi):

- **Sensori di pista:** 19 sensori di distanza dal bordo pista (trackSensors[0-18])
- **Sensori di movimento:** velocità longitudinale (speedX), velocità laterale (lateralSpeed), velocità verticale (zSpeed)
- **Sensori di posizione:** angolo rispetto all'asse pista (angleToTrackAxis), posizione laterale (trackPosition), altezza (z), distanza dalla linea di partenza (distanceFromStartLine), distanza percorsa (distanceRaced)
- **Sensori di stato veicolo:** RPM motore, livello carburante (fuelLevel), danni (damage), tempo giro corrente/precedente
- **Sensori focus:** 5 sensori direzionali per rilevamento ostacoli (focusSensors[0-4])
- **Sensori ruote:** 4 valori di velocità di rotazione ruote (wheelSpinVelocity[0-3])
- **Azioni di controllo:** sterzo, accelerazione, frenata, marcia, frizione, velocità target

Caratteristiche principali:

- **Completezza sensoriale:** Include tutti i sensori TORCS disponibili (esclusi solo sensori avversari)
- **Flessibilità di sviluppo:** Permette di sperimentare con diverse combinazioni di feature senza dover registrare nuovamente i dati
- **Analisi esplorativa:** Facilita l'analisi della correlazione tra diversi sensori e le azioni di guida
- **Backup completo:** Mantiene tutte le informazioni disponibili per future modifiche ai modelli

Vantaggi strategici:

- **Efficienza temporale:** Evita la necessità di registrare nuovamente sessioni di guida in caso di cambiamenti nelle feature utilizzate
- **Sperimentazione rapida:** Consente di testare rapidamente l'impatto di diverse combinazioni di sensori
- **Analisi retrospettiva:** Permette di analizzare l'importanza di sensori inizialmente scartati

4.1.2 Human Dataset

L'`human_dataset.csv` rappresenta il dataset ottimizzato che contiene esclusivamente i sensori effettivamente utilizzati dai modelli di machine learning implementati.

Sensori memorizzati (totale: 14 feature + 3 azioni):

- **Sensori di pista ottimizzati:** 10 sensori di distanza selezionati con pattern alternato (`trackSensors[0,2,4,6,8,10,12,14,16,18]`) per copertura ottimale del campo visivo
- **Sensori di movimento:** velocità longitudinale (`speedX`)
- **Sensori di posizione:** angolo rispetto all'asse pista (`angleToTrackAxis`), posizione laterale (`trackPosition`)
- **Sensore di riferimento:** distanza dalla linea di partenza (`distanceFromStartLine`)
- **Azioni di controllo:** sterzo (-1.0 a 1.0), accelerazione (0.0 a 1.0), frenata (0.0 a 1.0)

Caratteristiche principali:

- **Ottimizzazione computazionale:** Include solo le 14 feature più rilevanti per i modelli finali
- **Efficienza di caricamento:** Dimensioni ridotte per tempi di caricamento più rapidi
- **Compatibilità diretta:** Formato direttamente utilizzabile dai driver KNN e MLP
- **Preprocessing integrato:** Dati già processati e normalizzati per l'utilizzo immediato
- **Selezione intelligente:** Pattern alternato dei sensori di pista per massimizzare la copertura informativa

4.1.3 Considerazioni sui Sensori Utilizzati

La selezione dei sensori per il dataset presenta implicazioni significative per la generalizzabilità del modello:

Sensore Distanza dalla Linea di Partenza

L'inclusione del sensore `distanceFromStartLine` tra le feature rappresenta una scelta progettuale consapevole con importanti conseguenze:

- **Localizzazione Specifica:** Questo sensore fornisce informazioni sulla posizione dell'auto nel circuito, permettendo di associare determinate azioni a punti specifici della pista
- **Specializzazione del Dataset:** Il dataset registra informazioni ottimizzate per un particolare circuito, rendendo le azioni apprese specifiche per quella configurazione di pista
- **Limitazioni di Generalizzazione:** Il modello risultante potrebbe non adattarsi efficacemente a circuiti diversi da quello utilizzato per il training

- **Problematiche Multi-Circuito:** L'inclusione di dati da circuiti diversi nello stesso dataset potrebbe causare conflitti nelle associazioni posizione-azione, degradando le performance

Motivazioni della Scelta

Questa decisione è stata presa coscientemente considerando i requisiti del progetto:

- **Requisito Specifico:** Il progetto richiedeva che l'auto guidasse efficacemente su un circuito specifico
- **Ottimizzazione Performance:** La specializzazione per un singolo circuito permette performance superiori rispetto a un approccio generico
- **Semplicità Implementativa:** Evita la complessità di gestire variazioni tra circuiti diversi

Approccio Alternativo per Modelli Generici

Per implementare un modello più generico capace di operare su circuiti multipli, sarebbe necessario:

- **Rimozione Sensore Posizionale:** Escludere `distanceFromStartLine` dalle feature
- **Focus su Sensori Locali:** Utilizzare esclusivamente sensori che descrivono l'ambiente immediato (sensori di pista, velocità, angolo, posizione laterale)
- **Invarianza Geometrica:** Basare le decisioni solo su caratteristiche geometriche locali indipendenti dalla posizione assoluta
- **Dataset Multi-Circuito:** Raccogliere dati da circuiti diversi per aumentare la diversità delle situazioni di training

4.2 Pipeline di Conversione

Il sistema implementa una pipeline automatica di conversione tra i due formati:

1. **Raccolta:** L'`EnhancedDataCollector` registra tutti i sensori nell'enhanced dataset
2. **Selezione:** Un processo di feature selection identifica i sensori più rilevanti
3. **Conversione:** Il sistema di menu include strumenti per convertire automaticamente da enhanced a human dataset
4. **Validazione:** Controlli di integrità verificano la correttezza della conversione

4.3 Preprocessing dei Dati

Entrambi i dataset subiscono processi di preprocessing specifici:

- **Normalizzazione:** I valori dei sensori vengono normalizzati per migliorare la convergenza dei modelli
- **Gestione valori anomali:** Filtri per identificare e gestire letture sensoriali errate
- **Sincronizzazione temporale:** Allineamento tra stato sensoriale e azioni umane
- **Discretizzazione:** Per il KNN Classifier, conversione delle azioni continue in classi discrete

4.4 Sfide del Bilanciamento del Dataset

Uno degli aspetti più critici emersi durante lo sviluppo del progetto è stata la realizzazione di un dataset bilanciato e rappresentativo. Le difficoltà principali sono risultate strettamente correlate alle caratteristiche specifiche dei circuiti utilizzati per la raccolta dati.

4.4.1 Problematiche Legate ai Circuiti

La composizione del dataset è fortemente influenzata dalla geometria e dalle caratteristiche dei circuiti:

- **Predominanza di rettilinei:** Circuiti con lunghi tratti rettilinei generano una sovrarappresentazione di dati relativi alla guida in linea retta, creando uno sbilanciamento verso azioni di accelerazione e sterzo neutro
- **Asimmetria delle curve:** Tracciati con prevalenza di curve verso destra o verso sinistra introducono bias direzionali che compromettono la capacità del modello di generalizzare in entrambe le direzioni
- **Variabilità delle sezioni:** La mancanza di diversità nelle tipologie di curve (strette, ampie, chicane, esse) limita l'esposizione del modello a situazioni di guida variegata
- **Distribuzione delle velocità:** Circuiti con caratteristiche omogenee non forniscono una distribuzione equilibrata delle velocità di percorrenza

4.4.2 Impatto sui Modelli di Machine Learning

Questi sbilanciamenti si traducono in comportamenti problematici durante la guida autonoma:

- **Bias direzionale:** Tendenza dell'auto a privilegiare una direzione di sterzata rispetto all'altra
- **Gestione inadeguata delle curve:** Difficoltà nell'affrontare tipologie di curve sotto-rappresentate nel dataset

- **Velocità inappropriate:** Scelta di velocità non ottimali per sezioni di pista diverse da quelle predominanti nel training
- **Mancanza di adattabilità:** Ridotta capacità di adattamento a circuiti con caratteristiche diverse da quelli di training

4.4.3 Strategie di Bilanciamento Implementate

Per affrontare queste problematiche, è stato necessario dedicare una parte significativa del tempo di sviluppo all'analisi e al bilanciamento del dataset:

- **Analisi distributiva:** Utilizzo di strumenti di analisi Python per identificare sbilanciamenti nelle distribuzioni delle azioni e degli stati
- **Raccolta mirata:** Sessioni di guida specifiche su circuiti complementari per compensare le carenze identificate
- **Campionamento strategico:** Tecniche di under-sampling e over-sampling per equilibrare le diverse categorie di situazioni
- **Validazione incrociata:** Test su circuiti diversi da quelli di training per verificare la generalizzazione
- **Monitoraggio continuo:** Sistema di analisi integrato per valutare la qualità e il bilanciamento dei dati durante la raccolta

4.5 Sistema di Analisi Python

È stato sviluppato un sistema di analisi dedicato che ha facilitato significativamente il processo di bilanciamento:

- **Visualizzazione distributiva:** Grafici e statistiche per identificare rapidamente sbilanciamenti
- **Analisi correlazionale:** Studio delle relazioni tra sensori e azioni per identificare pattern problematici
- **Metriche di qualità:** Indicatori quantitativi per valutare l'equilibrio del dataset
- **Strumenti di preprocessing:** Funzioni automatizzate per il bilanciamento e la pulizia dei dati

Questo approccio sistematico al bilanciamento del dataset, rispetto al fine-tuning dei parametri degli algoritmi, si è rivelato più determinante per le performance finali dei modelli, evidenziando l'importanza cruciale della qualità dei dati nell'apprendimento automatico.

4.5.1 Sistema di Fine Tuning

Per ottimizzare al meglio le prestazioni dei modelli è stato sviluppato un sistema di fine-tuning basato sul modulo GridSearchCV della libreria Scikit-learn, con lo scopo di ricercare i migliori iperparametri. Questo approccio non solo ha permesso di risparmiare tempo, ma anche di procedere in maniera non puramente sperimentale.

4.5.2 Librerie Utilizzate

Per la realizzazione del sistema di analisi, sono state adottate le seguenti librerie, ciascuna con un ruolo specifico:

- **Pandas:** per la gestione e la manipolazione dei dati tabellari. Ha permesso di caricare e trasformare il dataset in modo intuitivo ed efficiente.
- **Scikit-learn:** impiegata soprattutto per il fine-tuning dei modelli tramite il modulo GridSearchCV. Sono state fondamentali le funzioni messe a disposizione che hanno permesso di calcolare e visualizzare in modo immediato le principali metriche di qualità.
- **Matplotlib:** per la visualizzazione grafica dei dati. Ha reso possibile rappresentare i dati in una forma intuitiva e immediata, supportando l'identificazione di sbilanciamenti e anomalie.
- **Jupyter Notebook:** il formato `.ipynb` è stato utilizzato per sviluppare e testare il codice in maniera interattiva, permettendo una visualizzazione immediata dei dati, delle trasformazioni applicate e dei risultati.

I principali parametri

5 Metodologia di Sperimentazione

La metodologia di sperimentazione adottata per questo progetto si è basata su un approccio iterativo e data-driven, focalizzato sull'ottimizzazione delle performance dei modelli attraverso l'analisi approfondita delle distribuzioni dei dati e il bilanciamento strategico del dataset.

5.1 Approccio Sperimentale

Prima di procedere con la valutazione euristica dei modelli su uno specifico dataset, è stata effettuata un'analisi approfondita delle feature selezionate e della loro distribuzione. Il primo modello sviluppato è stato il KNN regressivo, che presenta il vantaggio di adattarsi fortemente ai dati di input.

5.2 Analisi delle Feature

Una serie di grafici ha permesso di identificare le feature più discriminanti nei vari scenari. Le feature selezionate sono riportate nel grafico in figura 1.

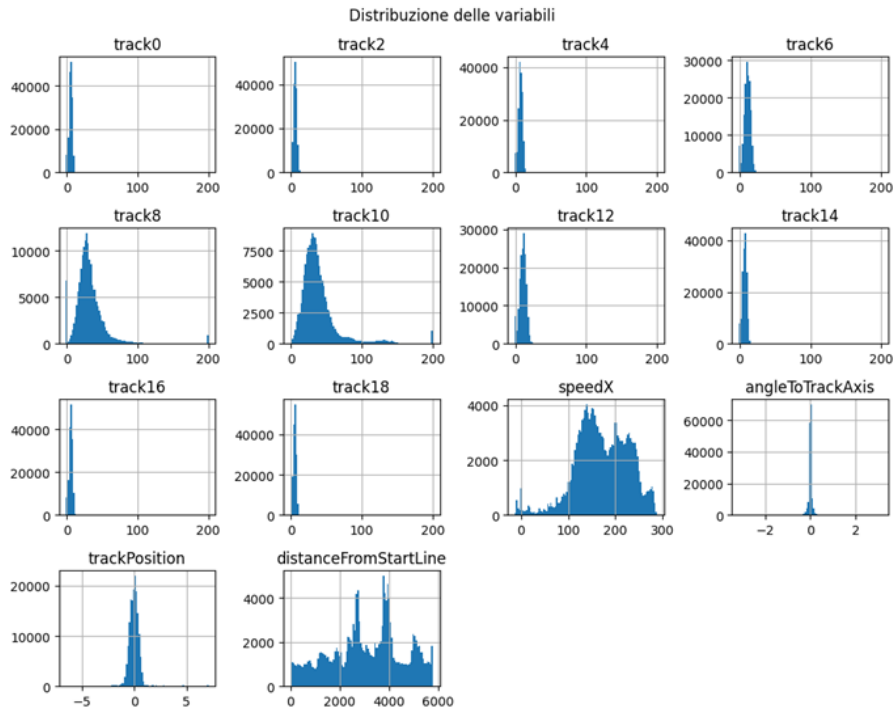


Figura 1: Feature selezionate per i modelli di machine learning

5.3 Problematiche Iniziali del KNN

Grazie alla natura data-driven del KNN, è stato osservato che, nelle prime prove, indipendentemente dai parametri utilizzati (purché ragionevoli), il veicolo tendeva a sterzare verso destra anche nei tratti rettilinei, mostrando difficoltà nell'affrontare curve a sinistra.

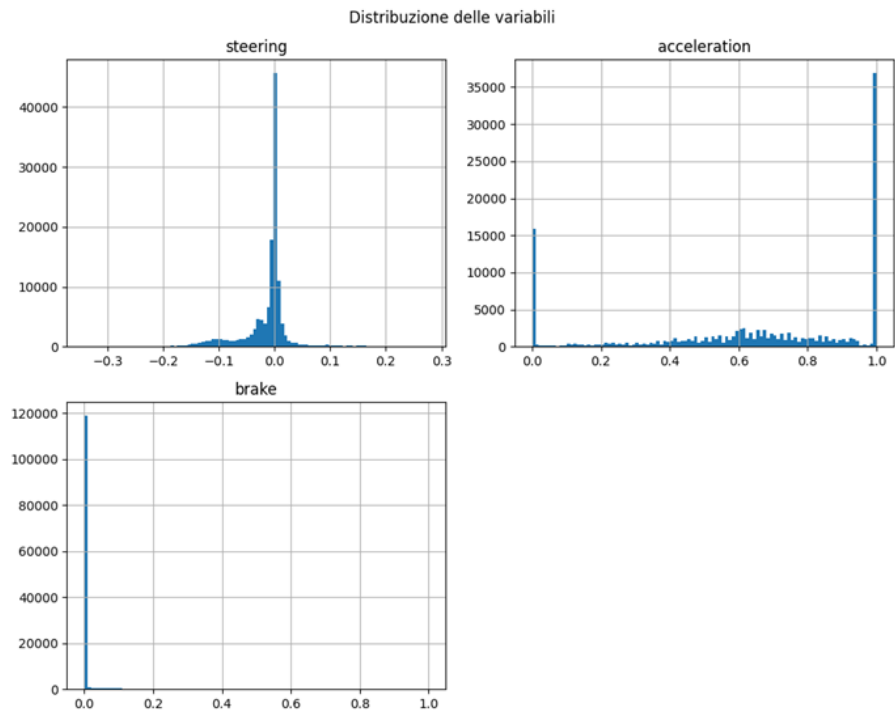


Figura 2: Comportamento iniziale del KNN con bias direzionale

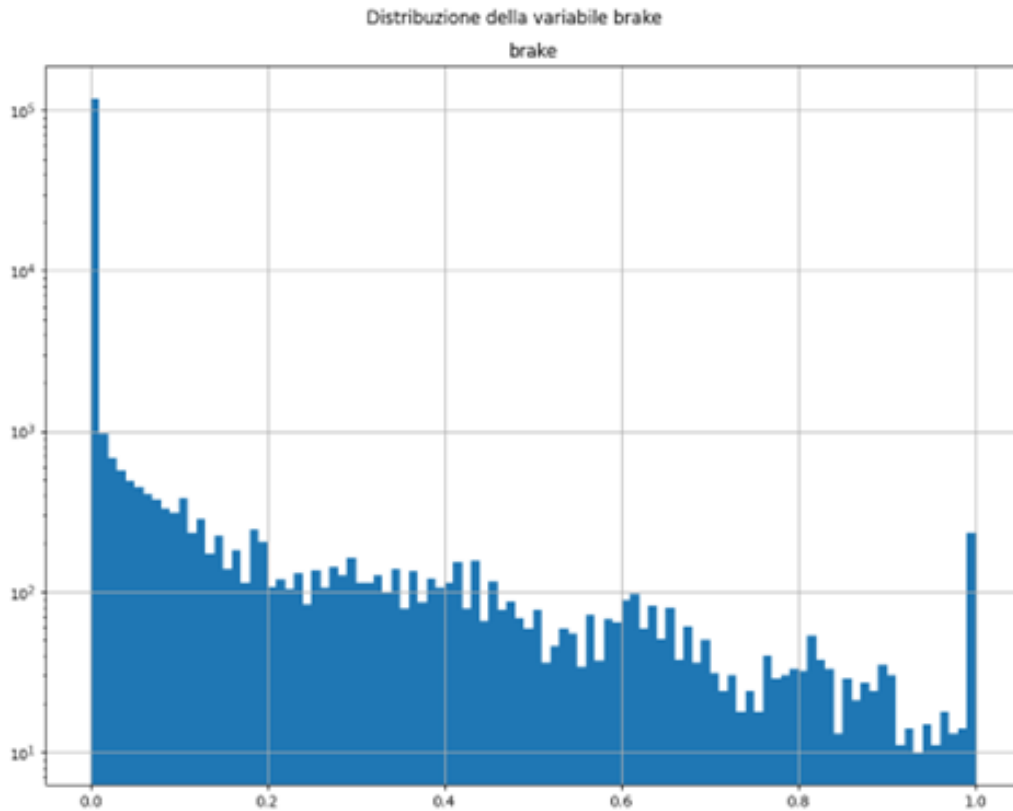


Figura 3: Distribuzione iniziale dei dati di steering e brake

6 Risultati

6.1 Analisi della Distribuzione Iniziale

Questa problematica non è risultata sorprendente: come evidenziato dal grafico relativo a uno dei dataset iniziali (pur contenendo un numero sufficiente di dati), i valori dello steering si concentrano prevalentemente attorno allo zero (sterzo dritto) o su valori negativi (sterzo inclinato a destra), mentre i valori positivi (sterzo a sinistra) sono meno frequenti. Inoltre, la distribuzione del brake mostra valori prossimi allo zero, suggerendo uno stile di guida poco orientato alla frenata.

6.2 Influenza delle Caratteristiche del Tracciato

Queste distribuzioni sono influenzate dalle caratteristiche intrinseche del tracciato, che risulta veloce, con poche curve e prevalentemente sterzate a destra. Le opzioni disponibili erano due:

1. Aggiungere un elevato numero di dati, accettando un possibile overfitting, che in questo caso specifico poteva essere vantaggioso, ma con l'inconveniente di tempi computazionali elevati.
2. Ridurre il dataset, sacrificando parte dell'informazione per ottenere una distribuzione più equilibrata.

6.3 Strategia di Bilanciamento Adottata

È stata scelta la seconda opzione, ottenendo una distribuzione più regolare:

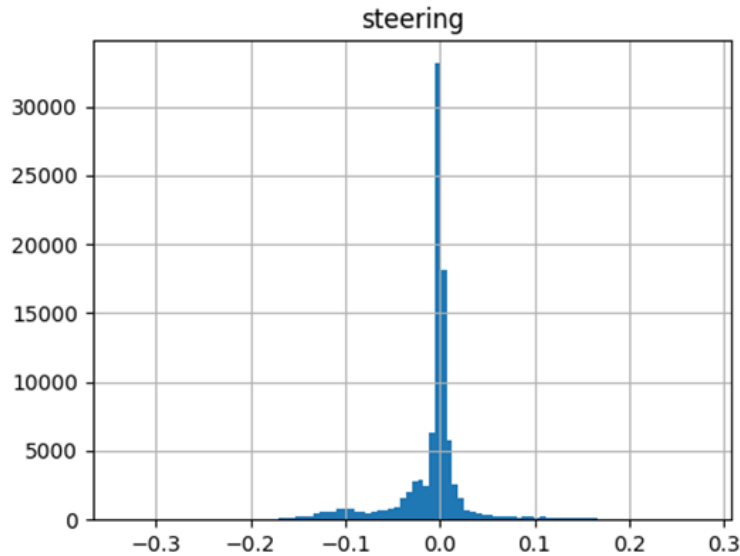


Figura 4: Distribuzione bilanciata dopo il preprocessing

Dopo il bilanciamento, la distribuzione risultava composta da circa il 10% di dati con sterzo dritto e il 45% rispettivamente con sterzo a destra e a sinistra. Con questa configurazione, il veicolo ha iniziato a gestire correttamente le curve. Tuttavia, la riduzione del dataset ha comportato una perdita di informazioni, rendendo il numero di dati insufficiente.

6.4 Ottimizzazione Iterativa del KNN

Progressivamente sono stati aggiunti dati specifici relativi alle curve, dove il modello mostrava maggiori difficoltà, fino a ottenere un giro completo con il KNN regressivo. Considerando che le prestazioni del KNN dipendono fortemente dall'hardware utilizzato, è stato necessario raccogliere ulteriori dati, ribilanciarli e determinare il numero massimo di campioni analizzabili senza compromettere le performance. Sono stati quindi aggiunti dati fino al limite computazionale consentito, includendo anche situazioni di recupero da uscite di pista.

6.4.1 Confronto tra modelli KNN

Dopo aver creato e analizzato il dataset, è stata utilizzata la tecnica GridSearchCV in Python per identificare i parametri ottimali, come la normalizzazione e il valore di K, al fine di simulare e valutare il comportamento del modello in diverse configurazioni.

In fase di valutazione è stata inoltre applicata la regola dell'80/20, suddividendo il dataset in 80% per l'addestramento e 20% per il test. Questo approccio consente di garantire una buona rappresentatività dei dati durante l'addestramento, mantenendo al contempo un set di test sufficientemente ampio per valutare le capacità di generalizzazione del modello. Per implementare questa suddivisione è stato utilizzato lo ShuffleSplit con i seguenti parametri: `ShuffleSplit(n_splits=10, train_size=0.8, test_size=0.2, random_state=42)`.

La GridSearchCV consente di testare sistematicamente diverse combinazioni di iperparametri, fornendo una serie di metriche utili per valutare la qualità dei modelli. Di seguito vengono descritti i principali parametri e risultati restituiti:

- `param_model__n_neighbors`: indica il numero di vicini (`n_neighbors`) considerati in ciascuna configurazione.
- `split<n>_test_score`: mostra il punteggio ottenuto dal modello su ciascun fold di test durante la validazione incrociata. Ad esempio, `split0_test_score` rappresenta il punteggio sul primo fold, `split1_test_score` sul secondo, e così via.
- `mean_test_score`: rappresenta la media dei punteggi ottenuti sui vari fold di test per una specifica combinazione di iperparametri. È un indicatore chiave delle prestazioni generali del modello.
- `std_test_score`: indica la deviazione standard dei punteggi sui diversi fold. Un valore basso suggerisce che il modello è stabile e coerente nelle sue prestazioni.
- `rank_test_score`: classifica le combinazioni di iperparametri in base al `mean_test_score`. Il rango 1 corrisponde alla combinazione con il punteggio medio più elevato.
- `mean_fit_time`: tempo medio impiegato per addestrare il modello su tutti i fold di training.
- `std_fit_time`: deviazione standard del tempo di addestramento.
- `mean_score_time`: tempo medio necessario per calcolare il punteggio sul set di test.
- `std_score_time`: deviazione standard del tempo di calcolo del punteggio.

La combinazione ottimale dovrebbe presentare un `mean_test_score` elevato e una `std_test_score` contenuta. Tuttavia, nel progetto è stato scelto un valore di K diverso da 2 (che, secondo il grafico, risulta il migliore). Questo perché, per valori di K troppo piccoli ($K < 7$), il modello tendeva a non generalizzare correttamente, mostrando una media leggermente migliore nei test ma prestazioni scarse nella pratica. Al contrario, per valori di K troppo grandi ($K > 10$), il modello perdeva capacità discriminante, con il `mean_test_score` che tendeva asintoticamente a zero, comportandosi come se stesse calcolando una media tra tutti i valori.

Pertanto, tra i valori intermedi, è stato selezionato il valore di K con la deviazione standard più bassa, compatibile con le risorse computazionali disponibili, senza perdita significativa di informazione.

Dal grafico si osserva inoltre che la normalizzazione personalizzata applicata ai parametri risulta linearmente più efficace rispetto ad altre tecniche comunemente utilizzate, come `StandardScaler` e `QuantileTransformer`.

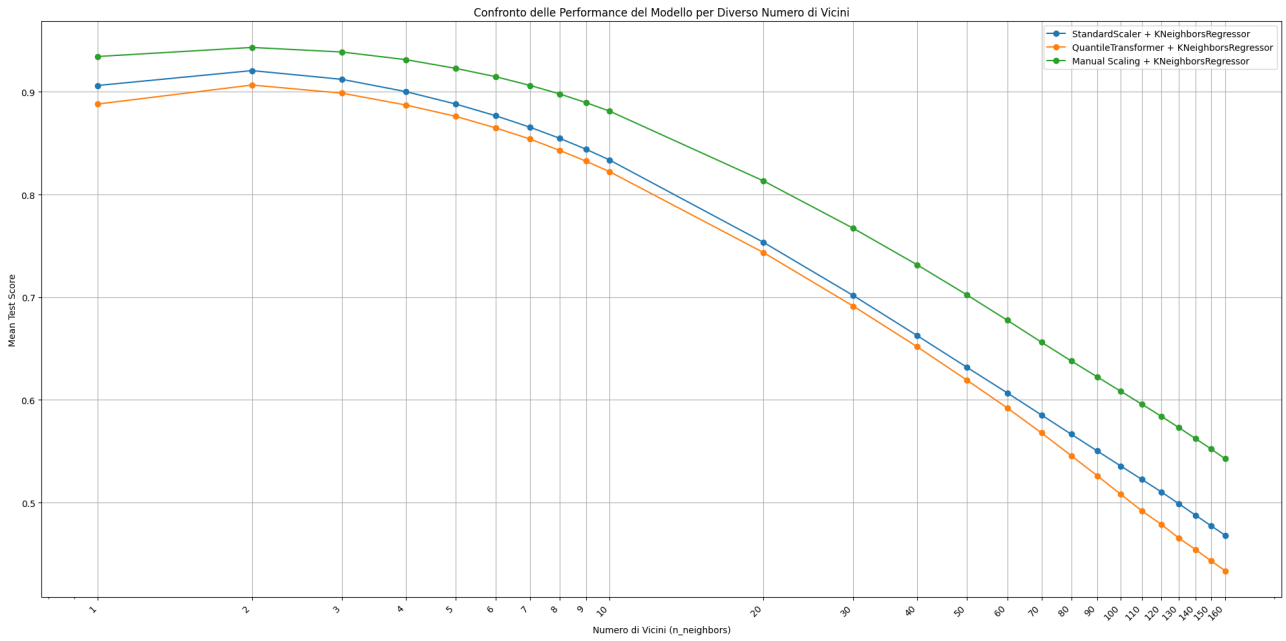


Figura 5: Risultati del confronto tra modelli KNN

6.5 Limitazioni Hardware

Questo significa che, cambiando calcolatore, il modello potrebbe non funzionare correttamente, ma è stato comunque ottimizzato per il limite fisico disponibile.

6.6 Sviluppo del Modello MLP

Dopo l'esperienza acquisita con il KNN, è stato sviluppato anche un modello MLP regressivo, per il quale sono stati raccolti ulteriori dati, includendo stili di guida che sfruttano maggiormente la frenata. In questo caso, il freno non è utilizzato solo per rallentare con lo sterzo dritto, ma anche per effettuare micro-correzioni durante la percorrenza delle curve. Queste correzioni riducono la velocità in curva, ma migliorano la traiettoria in uscita.

6.7 Bilanciamento Dataset per MLP

Il bilanciamento del dataset per l'MLP è stato leggermente modificato: sono stati rimossi solo i dati con $\text{brake} = 0$, preservando quelli relativi alla frenata, già di per sé poco frequenti.

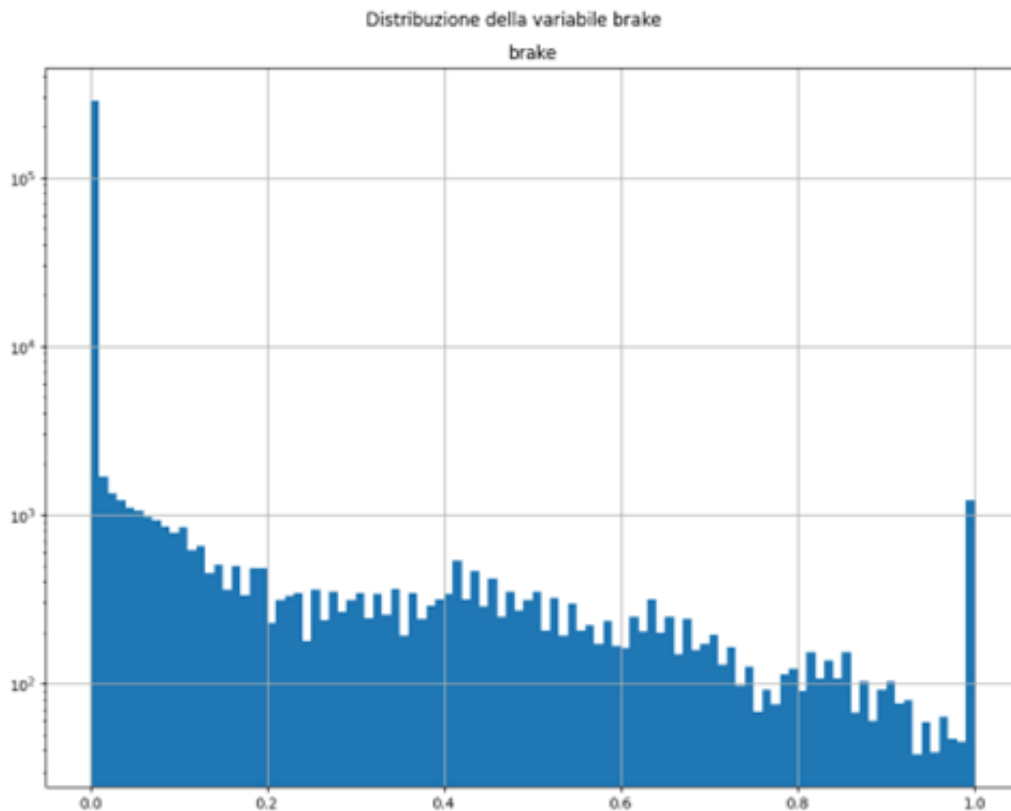


Figura 6: Distribuzione dei dati per il modello MLP con preservazione dei dati di frenata

6.8 Addestramento MLP

L'uso della grid search ha permesso di testare sistematicamente diverse combinazioni di iperparametri, fornendo al contempo una serie di metriche utili per valutare la qualità dei modelli. Ciononostante, la scelta dei parametri non è stata solo frutto della grid search, ma è stata indirizzata anche dalle nostre esperienze.

Facendo riferimento ai listati di codice, segue la descrizione dei parametri più importanti :

- `mlp_hidden_layer_sizes`: definisce la struttura dei livelli nascosti.
- `mlp_activation`: definisce la funzione di attivazione da utilizzare.
- `mlp_solver`: indica l'algoritmo utilizzato per l'ottimizzazione dei pesi.
- `mlp_alpha`: indica il valore da assegnare al parametro di regolarizzazione.
- `mlp_learning_rate`: definisce l'andamento del learning rate.

```

1 paramGrid = {
2     "mlp__hidden_layer_sizes": [(12), (6,6), (8,4), (12,8,4), (32,16,8)],
3     "mlp__activation": ["relu"],
4     "mlp__solver": ["adam"],
5     "mlp__alpha": [0.01,0.001],
6     "mlp__learning_rate": ["constant","adaptive"],
7 }

```

Listing 1: paramGrid usata per l'addestramento dell'MLP

- **scoring**: indica quali parametri di qualità valutare durante la fase di test, per ogni combinazione.
- **neg_mean_squared_error**: errore quadratico medio cambiato di segno. Indica quanto le predizioni si discostano dalla realtà.
- **r2**: coefficiente di determinazione. Misura quanto le predizioni del modello riescono a catturare le variazioni reali dei dati.
- **refit**: definisce su quale metrica basarsi per scegliere il modello finale.

```

1 cv_splitter = ShuffleSplit(n_splits=5, test_size=0.20, random_state=42)
2
3 gridsearch = GridSearchCV(estimator=pipeline,
4                           param_grid=paramGrid,
5                           scoring=["neg_mean_squared_error", "r2"],
6                           refit="neg_mean_squared_error",
7                           cv=cv_splitter,
8                           verbose=2,
9                           n_jobs=-1)

```

Listing 2: modulo GridSearchCV

Per quanto riguarda la struttura dei livelli nascosti, ci si è prima basati su delle regole euristiche come la regola dei 2/3, secondo cui il numero di neuroni nei livelli nascosti può essere vicino a due terzi della dimensione dell'input. In un secondo momento sono state sperimentate architetture con progressione decrescente (es. 32-16-8).

Il modello migliore prodotto dalla prima grid search dimostrava di avere un R^2 di 0.667258, un valore accettabile, che indicava che il modello fosse in grado di spiegare il suo comportamento circa il 67% delle volte. Ciononostante, una volta provato su Torcs, questo modello aveva delle performance non ottimali, probabilmente a causa del fatto che non generalizzasse abbastanza. Le cause della scarsa generalizzazione sono state individuate nel numero di livelli e neuroni troppo elevato e nel valore dell'alpha troppo basso. Sulla base di queste considerazioni è stato addestrato un modello su nuovi parametri :

- Il numero di neuroni nei livelli nascosti è stato portato a 32 per il primo livello e 16 per il secondo.

- Il valore dell'alpha è stato alzato da 0.001 a 0.01, con lo scopo di bilanciare meglio i pesi in modo da non mandare il modello in overfitting.

Una volta testato su Torcs, il nuovo modello si è dimostrato in grado di completare il giro della pista senza problemi, correggendo i suoi errori quando inevitabilmente ne commetteva uno.

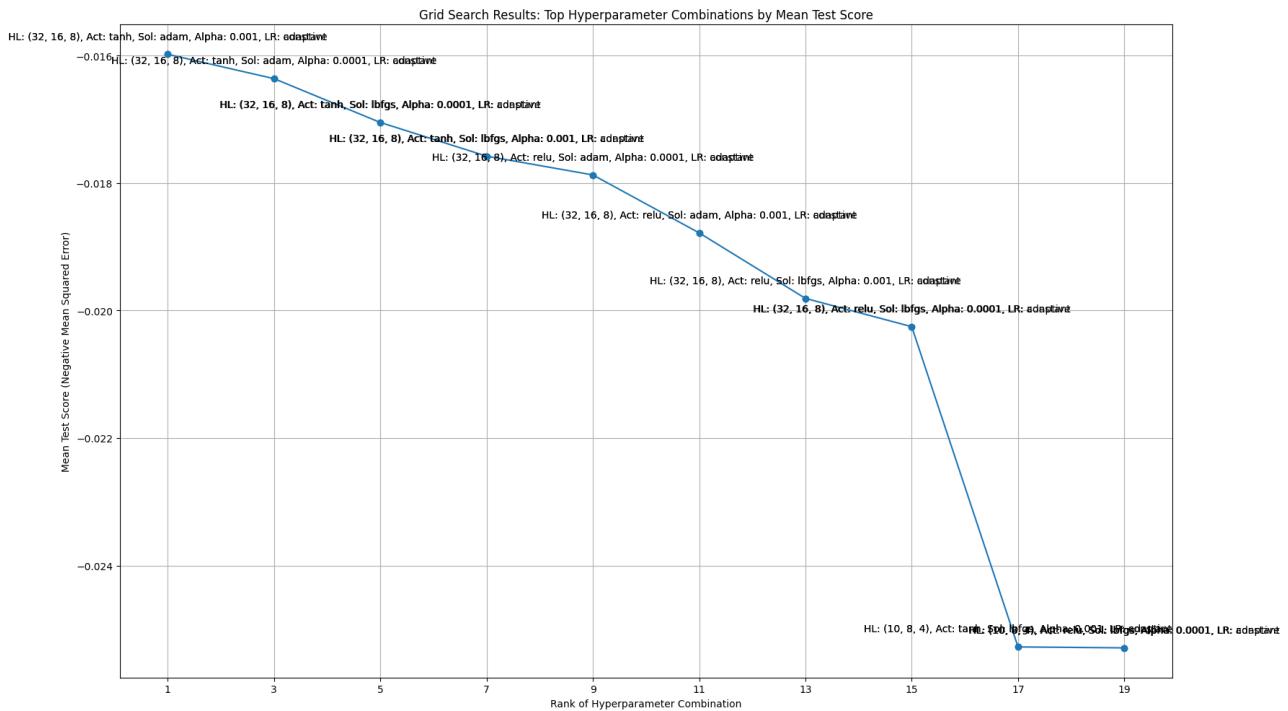


Figura 7: Risultati del confronto tra modelli MLP iniziali

7 Analisi delle Performance

7.1 Performance Computazionali del KNN

L'analisi delle performance ha evidenziato che le prestazioni del KNN dipendono fortemente dall'hardware utilizzato e dalla dimensione del dataset. I principali fattori limitanti identificati sono:

- **Caricamento dati:** Il tempo di caricamento del dataset CSV aumenta linearmente con la dimensione
- **Costruzione KD-Tree:** Complessità $O(n \log n)$ per la costruzione dell'albero
- **Ricerca KNN:** Complessità $O(\log n)$ per ogni query, ma con overhead significativo per dataset grandi
- **Memoria:** Caricamento completo del dataset in memoria RAM

7.2 Ottimizzazioni Implementate

Per mitigare i problemi di performance sono state implementate diverse ottimizzazioni:

- **Precaricamento:** Costruzione del KD-Tree durante l'inizializzazione per evitare latenze durante la guida
- **Normalizzazione efficiente:** Preprocessing dei dati per migliorare le performance di ricerca
- **Gestione timeout:** Implementazione di meccanismi per evitare blocchi del client
- **Dimensionamento adattivo:** Determinazione automatica del numero massimo di campioni gestibili

7.3 Confronto KNN vs MLP

Il confronto tra i due approcci ha evidenziato caratteristiche complementari:

KNN Regressivo:

- Adattamento forte ai dati di training
- Performance dipendenti dall'hardware
- Gestione efficace delle situazioni note
- Limitazioni con dataset molto grandi

MLP:

- Maggiore capacità di generalizzazione
- Performance più stabili su hardware diverso
- Gestione migliorata della frenata
- Tempi di inferenza più prevedibili

8 Discussione

8.1 Impatto del Bilanciamento del Dataset

L'esperienza di sviluppo ha dimostrato che il bilanciamento del dataset è più critico per le performance finali rispetto al fine-tuning dei parametri degli algoritmi. Le principali lezioni apprese includono:

- **Qualità vs Quantità:** Un dataset più piccolo ma bilanciato supera un dataset grande ma sbilanciato
- **Rappresentatività:** L'importanza di includere situazioni diverse e rappresentative
- **Analisi continua:** La necessità di monitoraggio costante delle distribuzioni durante la raccolta

8.2 Limitazioni e Trade-off

Il progetto ha evidenziato diversi trade-off fondamentali:

- **Performance vs Accuratezza:** Limitazioni hardware richiedono compromessi sulla dimensione del dataset
- **Generalizzazione vs Specializzazione:** Modelli ottimizzati per circuiti specifici vs capacità di adattamento
- **Complessità vs Manutenibilità:** Bilanciamento tra sofisticazione del modello e facilità di gestione

8.3 Approcci Metodologici

L'approccio iterativo adottato si è rivelato efficace, permettendo di:

- Identificare rapidamente problematiche nei dati
- Implementare correzioni mirate
- Validare incrementalmente le migliorie
- Ottimizzare le risorse computazionali disponibili

9 Conclusioni

9.1 Valutazione Comparativa dei Modelli

Dalla sperimentazione condotta, è emersa una chiara gerarchia di performance tra i tre modelli implementati:

- **KNN Classificatore:** Si è dimostrato il meno adatto al problema della guida autonoma. La discretizzazione delle azioni di controllo introduce limitazioni significative nella fluidità della guida, risultando in comportamenti meno naturali e performance inferiori. Le motivazioni principali includono:
 - La perdita di informazione dovuta alla quantizzazione delle azioni continue
 - Transizioni brusche tra classi discrete che compromettono la smoothness della guida
 - Difficoltà nel rappresentare adeguatamente la varietà di situazioni di guida con un numero limitato di classi
 - Maggiore sensibilità al rumore nei dati di training
- **KNN Regressivo:** Ha mostrato risultati significativamente migliori, dimostrando l'efficacia dell'approccio regressivo per la predizione continua delle azioni di controllo. La capacità di produrre output continui si traduce in una guida più fluida e naturale. I vantaggi osservati sono:

- Preservazione della continuità nelle azioni di controllo (sterzo, accelerazione, frenata)
 - Capacità di interpolare efficacemente tra situazioni simili nel dataset
 - Comportamento più naturale e prevedibile del veicolo
 - Gestione implicita delle situazioni critiche tramite i dati di recupero nel dataset
 - **Limitazione di scalabilità:** Tuttavia, il KNN presenta vincoli significativi con dataset di grandi dimensioni, dovendo esplorare il KD-tree in tempo reale per ogni predizione, il che può causare timeout e compromettere i tempi di risposta richiesti dal simulatore
- **MLP Regressivo:** Si è rivelato leggermente superiore al KNN regressivo in termini di performance, mostrando una migliore capacità di generalizzazione e gestione delle situazioni complesse. Tuttavia, questa superiorità viene bilanciata da una maggiore complessità implementativa e computazionale. Le ragioni della sua efficacia includono:
 - Capacità di apprendere relazioni non lineari complesse tra sensori e azioni
 - Migliore gestione della frenata grazie alla capacità di modellare interazioni complesse
 - Maggiore robustezza al rumore nei dati di input
 - Performance più stabili su hardware diverso e tempi di inferenza prevedibili
 - **Scalabilità superiore:** A differenza del KNN che deve esplorare il KD-tree in tempo reale rischiando timeout con dataset grandi, l'MLP effettua il processo di apprendimento offline e può quindi basarsi su quantità considerevolmente maggiori di dati senza impatti sui tempi di risposta

In sintesi, entrambi i modelli regressivi (KNN e MLP) si sono dimostrati significativamente più efficaci dell'approccio classificatorio, con l'MLP che presenta un leggero vantaggio a fronte di una maggiore complessità di implementazione.

9.2 Considerazioni Finali

L'esperienza di sviluppo ha confermato l'importanza cruciale della qualità dei dati nell'apprendimento automatico. Il successo del progetto è stato determinato più dalla cura nella preparazione del dataset che dalla sofisticazione degli algoritmi utilizzati, evidenziando come un approccio metodico e data-driven sia fondamentale per ottenere risultati efficaci in applicazioni di machine learning per la guida autonoma.